

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА
ФРАНКА

Факультет електроніки та комп'ютерних технологій

Звіт

про виконання лабораторної роботи №6
“КІЛЬКІСНА ОЦІНКА ІНФОРМАЦІЇ”

Виконав:

Студент 3 курсу

групи Фел-33

Абдулханов Абдул-Рахім

Викладач:

проф. Половинко І. І.

Львів-2026

Мета роботи: дослідити кількісні міри інформації, вивчити властивості власної інформації подій та ентропії Шеннона для дискретних джерел повідомлень, промодельовати оцінку ентропії та надлишковості випадкових сигналів і цифрових зображень з різними законами розподілу ймовірностей.

Теоретичні відомості:

Оцінка інформаційного змісту повідомлень і сигналів ґрунтується на імовірнісній природі джерела даних. Якщо випадкова подія E відбувається з апіорною ймовірністю $P(E)$, то згідно з теорією К. Шеннона кількість інформації (власна інформація події) визначається як:

$$I(E) = -\log_2 P(E) \text{ [біт]}.$$

Власна інформація характеризує міру знятої невизначеності при здійсненні події. Для достовірної події ($P=1$) інформація дорівнює нулю $I(1)=0$, тоді як для малоімовірних подій ($P \rightarrow 0$) кількість інформації прямує до нескінченності. Якщо подія відповідає одному з двох рівноймовірних результатів ($P=0.5$), вона несе рівно 1 біт інформації.

Для дискретного джерела повідомлень з алфавітом $A = \{a_1, a_2, \dots, a_J\}$ та вектором ймовірностей появи символів $z = [P(a_1), \dots, P(a_J)]$ середня кількість інформації, що припадає на один згенерований символ, визначається ентропією Шеннона:

$$H(z) = -\sum_{j=1}^J P(a_j) * \log_2 P(a_j) \text{ [біт/символ]}.$$

Властивості ентропії Шеннона:

1. Невід'ємність: $H(z) \geq 0$. Ентропія дорівнює нулю тоді і тільки тоді, коли один із символів з'являється з імовірністю 1 (повна визначеність).
2. Максимальність: для алфавіту потужністю J ентропія досягає абсолютного максимуму при рівномірному розподілі ймовірностей символів $P(a_j) = 1/J$ для всіх j :

$$H_{\max} = \log_2(J) \text{ [біт/символ]}.$$

3. Надлишковість інформаційного джерела визначається коефіцієнтом:

$$R = 1 - (H / H_{\max}),$$

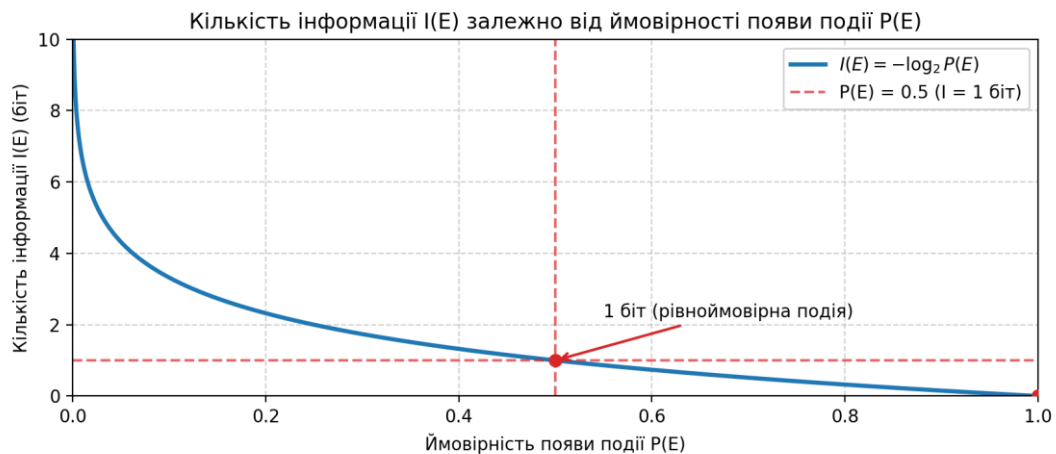
який показує частку невикористаної пропускну здатності та потенціал для стиснення даних без втрат (теорема Шеннона для кодування джерел).

У цифровій обробці зображень та сигналів (ЦОІ) гістограма яскравостей або амплітуд відліків використовується як оцінка розподілу ймовірностей першого порядку. Для 8-бітного монохромного зображення (256 градацій сірого) максимальна теоретична ентропія становить $H_{\max} = \log_2(256) = 8$ біт/піксель.

Виконання роботи

1. Побудова аналітичної залежності кількості власної інформації $I(E)$ від ймовірності появи події $P(E)$:

```
1 # -----
2 # 1. залежність власної інформації I(E) від ймовірності P(E)
3 # -----
4 p_vals = np.linspace(0.001, 1.0, 1000)
5 info_vals = -np.log2(p_vals)
6
7 plt.figure(figsize=(9, 4.8))
8 plt.plot(p_vals, info_vals, color="#1f77b4", linewidth=2.2, label=r"$I(E) = -\log_2 P(E)$")
9 plt.axvline(0.5, color="#d62728", linestyle="--", alpha=0.7, label="P(E) = 0.5 (I = 1 біт)")
10 plt.axhline(1.0, color="#d62728", linestyle="--", alpha=0.7)
11 plt.scatter([0.5, 1.0], [1.0, 0.0], color="#d62728", s=40, zorder=5)
12 plt.annotate("1 біт (підкидання монети)", xy=(0.5, 1.0), xytext=(0.55, 2.0),
13             arrowprops=dict(arrowstyle="→", color="#d62728", lw=1.5))
14 plt.title("Кількість інформації I(E) залежно від ймовірності появи події P(E)")
15 plt.xlabel("Ймовірність появи події P(E)")
16 plt.ylabel("Кількість інформації I(E) (біт)")
17 plt.xlim(0.0, 1.0)
18 plt.ylim(0.0, 10.0)
19 plt.grid(True, linestyle="--", alpha=0.6)
20 plt.legend(loc="upper right", framealpha=0.9)
21 plt.tight_layout()
22 plt.show()
```



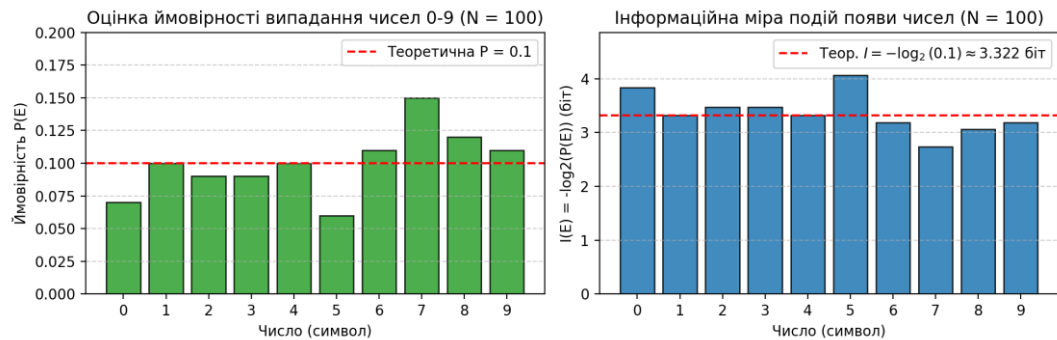
2. Генерація вибірки випадкових чисел у діапазоні 0-9 ($N = 100$), розрахунок емпіричних ймовірностей $P(E)$ та значень власної інформації $I(E)$:

```
1 # 2. вибірка випадкових чисел 0-9 для N=100
2 # -----
3 np.random.seed(42)
4 n100 = 100
5 a100 = np.random.randint(0, 10, size=n100)
6
7 counts100 = np.bincount(a100, minlength=10)
8 probs100 = counts100 / n100
9 infos100 = np.zeros(10)
10 for i, p in enumerate(probs100):
11     if p > 0:
12         infos100[i] = -np.log2(p)
13
14 print("---- Вибірка N = 100 елементів (числа 0-9) ----")
15 print("Згенерований масив:", a100.tolist())
16 print("Кількість появ кожного числа:", counts100.tolist())
17 print("Ймовірності P(E) та інформація I(E):")
18 for i in range(10):
19     print(f"Число {i}: кількість = {counts100[i]:2d}, P = {probs100[i]:.3f}, I = {infos100[i]:.3f} біт")
20
21 # графік ймовірностей
22 plt.figure(figsize=(9, 4.2))
23 plt.bar(range(10), probs100, color="#2ca02c", edgecolor="black", alpha=0.85)
24 plt.axhline(0.1, color="red", linestyle="--", label="Теоретична ймовірність P = 0.1")
25 plt.title("Оцінка ймовірності випадання чисел 0-9 (N = 100)")
26 plt.xlabel("Число (символ алфавіту джерела)")
27 plt.ylabel("Ймовірність P(E)")
28 plt.xticks(range(10))
29 plt.ylim(0, max(probs100) + 0.05)
30 plt.grid(axis="y", linestyle="--", alpha=0.7)
31 plt.legend(loc="upper right")
32 plt.tight_layout()
33 plt.show()
34
35 # графік інформаційної міри кожного числа
36 plt.figure(figsize=(9, 4.2))
37 plt.bar(range(10), infos100, color="#1f77b4", edgecolor="black", alpha=0.85)
38 plt.axhline(-np.log2(0.1), color="red", linestyle="--", label=r"Теоретична інформація $-\log_2(0.1) \approx 3.322$ біт")
39 plt.title("Інформаційна міра подій появи чисел (N = 100)")
40 plt.xlabel("Число")
41 plt.ylabel("I(E) = -log2(P(E)) (біт)")
42 plt.xticks(range(10))
43 plt.ylim(0, max(infos100) + 0.8)
44 plt.grid(axis="y", linestyle="--", alpha=0.7)
45 plt.legend(loc="upper right")
46 plt.tight_layout()
47 plt.show()
```

```

--- Вибірка N = 100 елементів (числа 0-9) ---
Кількість появ кожного числа: [7, 10, 9, 9, 10, 6, 11, 15, 12, 11]
Ймовірності P(E) та власна інформація I(E):
Число 0: кількість = 7, P = 0.070, I = 3.837 біт
Число 1: кількість = 10, P = 0.100, I = 3.322 біт
Число 2: кількість = 9, P = 0.090, I = 3.474 біт
Число 3: кількість = 9, P = 0.090, I = 3.474 біт
Число 4: кількість = 10, P = 0.100, I = 3.322 біт
Число 5: кількість = 6, P = 0.060, I = 4.059 біт
Число 6: кількість = 11, P = 0.110, I = 3.184 біт
Число 7: кількість = 15, P = 0.150, I = 2.737 біт
Число 8: кількість = 12, P = 0.120, I = 3.059 біт
Число 9: кількість = 11, P = 0.110, I = 3.184 біт

```



3. Дослідження залежності ентропії Шеннона $H(N)$ та надлишковості джерела R від розміру вибірки N у діапазоні від 50 до 5000:

```

1 # 3. залежність ентропії шеннона від розміру вибірки N
2 #
3 sample_sizes = [50, 100, 200, 500, 1000, 2000, 3000, 5000]
4 entropies = []
5 redundancies = []
6 h_max_10 = np.log2(10)
7
8 print("\n--- Дослідження ентропії Шеннона для різних розмірів вибірки ---")
9 print(f"Теоретичний максимум H_max = log2(10) = {h_max_10:.4f} біт")
10
11 for n in sample_sizes:
12     arr = np.random.randint(0, 10, size=n)
13     counts = np.bincount(arr, minlength=10)
14     p = counts / n
15     # шеннонівська ентропія
16     h = -np.sum([pi * np.log2(pi) for pi in p if pi > 0])
17     # надлишковість джерела
18     r = 1.0 - (h / h_max_10)
19     entropies.append(h)
20     redundancies.append(r)
21     print(f"N = {n:4d} | Ентропія H = {h:.4f} біт | Надлишковість R = {r * 100:.2f}%")
22
23 plt.figure(figsize=(9, 4.8))
24 plt.plot(sample_sizes, entropies, marker="o", color="#d62728", linewidth=2, label="Емпірична ентропія H(N)")
25 plt.axhline(h_max_10, color="black", linestyle="--", linewidth=1.5, label=f"H_max = log2(10) = {h_max_10:.4f} біт")
26 plt.title("Залежність ентропії джерела від розміру вибірки N")
27 plt.xlabel("Розмір вибірки N")
28 plt.ylabel("Ентропія Шеннона H (біт/символ)")
29 plt.ylim(3.1, 3.35)
30 plt.grid(True, linestyle="--", alpha=0.7)
31 plt.legend(loc="lower right")
32 plt.tight_layout()
33 plt.show()

```

```

--- Дослідження ентропії Шеннона для різних розмірів вибірки ---
Теоретичний максимум H_max = log2(10) = 3.3219 біт

```

N	Ентропія H	Надлишковість R
50	3.1772 біт	4.36%
100	3.2633 біт	1.76%
200	3.2911 біт	0.93%
500	3.3094 біт	0.38%
1000	3.3202 біт	0.05%
2000	3.3194 біт	0.08%
3000	3.3189 біт	0.09%
5000	3.3206 біт	0.04%

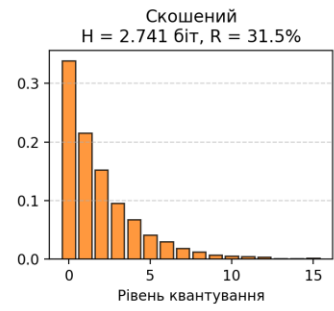
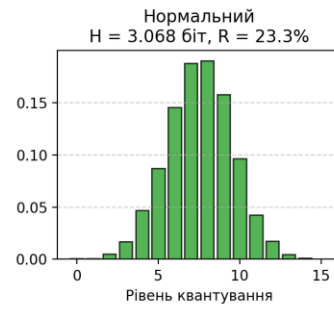
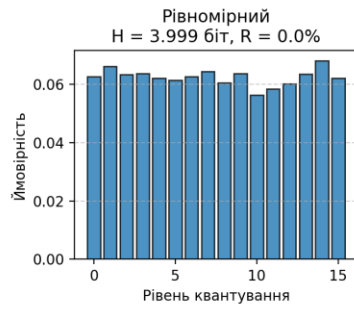
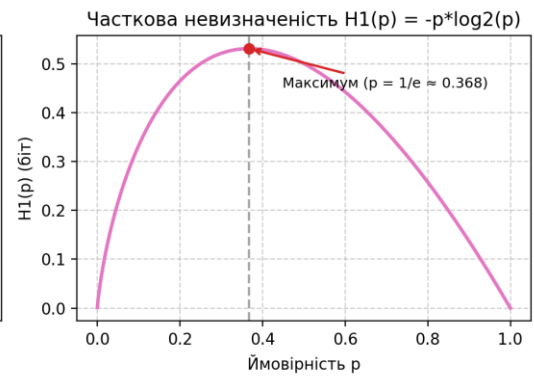
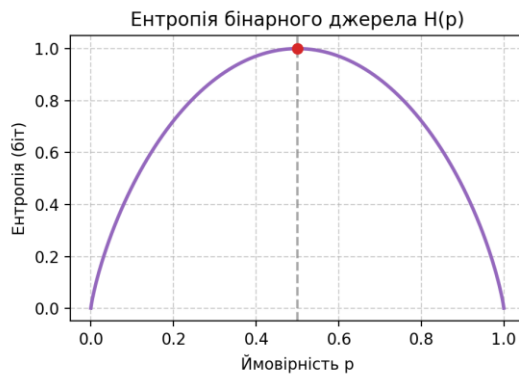


4. Аналіз ентропії бінарного джерела $H(p)$, часткової невизначеності $H_1(p)$ та порівняння ентропії сигналів з різними законами розподілу (рівномірний, нормальний, скошений):

```

1 # 4. ентропія сигналів із різними законами розподілу
2 # -----
3 # бінарне джерело та графік  $H(p)$ 
4 p_bin = np.linspace(0.0001, 0.9999, 500)
5 h_bin = -p_bin * np.log2(p_bin) - (1.0 - p_bin) * np.log2(1.0 - p_bin)
6 # функція невизначеності одного символу  $H_1(p) = -p \log_2(p)$ 
7 h1_single = -p_bin * np.log2(p_bin)
8
9 plt.figure(figsize=(10, 4.5))
10 plt.subplot(1, 2, 1)
11 plt.plot(p_bin, h_bin, color="#9467bd", linewidth=2.2)
12 plt.axvline(0.5, color="gray", linestyle="--", alpha=0.7)
13 plt.scatter([0.5], [1.0], color="#d62728", s=40, zorder=5)
14 plt.title("Ентропія бінарного джерела  $H(p)$ ")
15 plt.xlabel("Імовірність  $p$ ")
16 plt.ylabel("Ентропія (біт)")
17 plt.grid(True, linestyle="--", alpha=0.6)
18
19 plt.subplot(1, 2, 2)
20 plt.plot(p_bin, h1_single, color="#e377c2", linewidth=2.2)
21 p_max_h1 = 1.0 / np.e
22 h1_max = -p_max_h1 * np.log2(p_max_h1)
23 plt.axvline(p_max_h1, color="gray", linestyle="--", alpha=0.7)
24 plt.scatter([p_max_h1], [h1_max], color="#d62728", s=40, zorder=5)
25 plt.annotate(f"Максимум ( $p = 1/e \approx \{p\_max\_h1:.3f\}$ )", xy=(p_max_h1, h1_max),
26             xytext=(p_max_h1 + 0.08, h1_max - 0.08),
27             arrowprops=dict(arrowstyle="→", color="#d62728", lw=1.5))
28 plt.title("Часткова невизначеність  $H_1(p) = -p \log_2(p)$ ")
29 plt.xlabel("Імовірність  $p$ ")
30 plt.ylabel(" $H_1(p)$  (біт)")
31 plt.grid(True, linestyle="--", alpha=0.6)
32 plt.tight_layout()
33 plt.show()
34
35 # порівняння розподілів (рівномірний, нормальний, скошений/експоненційний)
36 n_samples = 10000
37 num_bins = 16
38 h_max_16 = np.log2(num_bins)
39
40 # 1) рівномірний
41 s_uniform = np.random.randint(0, num_bins, size=n_samples)
42 p_unif = np.bincount(s_uniform, minlength=num_bins) / n_samples
43 h_unif = -np.sum([p * np.log2(p) for p in p_unif if p > 0])
44 r_unif = 1.0 - h_unif / h_max_16
45
46 # 2) нормальний (дискретизований)
47 s_norm_raw = np.random.normal(loc=7.5, scale=2.0, size=n_samples)
48 s_norm = np.clip(np.round(s_norm_raw).astype(int), 0, num_bins - 1)
49 p_norm = np.bincount(s_norm, minlength=num_bins) / n_samples
50 h_norm = -np.sum([p * np.log2(p) for p in p_norm if p > 0])
51 r_norm = 1.0 - h_norm / h_max_16
52
53 # 3) скошений (експоненційний)
54 s_skew_raw = np.random.exponential(scale=2.5, size=n_samples)
55 s_skew = np.clip(np.floor(s_skew_raw).astype(int), 0, num_bins - 1)
56 p_skew = np.bincount(s_skew, minlength=num_bins) / n_samples
57 h_skew = -np.sum([p * np.log2(p) for p in p_skew if p > 0])
58 r_skew = 1.0 - h_skew / h_max_16
59
60 print("\n--- Порівняння ентропії для різних розподілів (M = 16 рівнів) ---")
61 print(f"Максимальна ентропія  $H_{max} = \log_2(16) = \{h\_max\_16:.4f\}$  біт")
62 print(f"1. Рівномірний розподіл:   H =  $\{h\_unif:.4f\}$  біт | Надлишковість R =  $\{r\_unif * 100:.2f\}\%$ ")
63 print(f"2. Нормальний розподіл:      H =  $\{h\_norm:.4f\}$  біт | Надлишковість R =  $\{r\_norm * 100:.2f\}\%$ ")
64 print(f"3. Скошений розподіл:        H =  $\{h\_skew:.4f\}$  біт | Надлишковість R =  $\{r\_skew * 100:.2f\}\%$ ")
65
66 plt.figure(figsize=(12, 4.2))
67 plt.subplot(1, 3, 1)
68 plt.bar(range(num_bins), p_unif, color="#1f77b4", edgecolor="black", alpha=0.8)
69 plt.title(f"Рівномірний  $nH = \{h\_unif:.3f\}$  біт, R =  $\{r\_unif*100:.1f\}\%$ ")
70 plt.xlabel("Рівень квантування")
71 plt.ylabel("Імовірність")
72 plt.grid(axis="y", linestyle="--", alpha=0.6)
73
74 plt.subplot(1, 3, 2)
75 plt.bar(range(num_bins), p_norm, color="#2ca02c", edgecolor="black", alpha=0.8)
76 plt.title(f"Нормальний  $nH = \{h\_norm:.3f\}$  біт, R =  $\{r\_norm*100:.1f\}\%$ ")
77 plt.xlabel("Рівень квантування")
78 plt.grid(axis="y", linestyle="--", alpha=0.6)
79
80 plt.subplot(1, 3, 3)
81 plt.bar(range(num_bins), p_skew, color="#ff7f0e", edgecolor="black", alpha=0.8)
82 plt.title(f"Скошений  $nH = \{h\_skew:.3f\}$  біт, R =  $\{r\_skew*100:.1f\}\%$ ")
83 plt.xlabel("Рівень квантування")
84 plt.grid(axis="y", linestyle="--", alpha=0.6)
85 plt.tight_layout()
86 plt.show()

```



5. Ентропійна оцінка цифрових 8-бітових зображень різної структури (шум, градієнт, двохрівневий узор) на основі гістограм яскравості:

```

1 # 5. оцінка ентропії цифрових зображень на основі гістограм
2 # -----
3 # створення трьох тестових 8-бітних зображень 128x128 (256 рівнів)
4 h_img, w_img = 128, 128
5 h_max_img = 8.0 # log2(256) біт на піксель
6
7 # а) випадковий шум (максимальна невизначеність)
8 img_noise = np.random.randint(0, 256, size=(h_img, w_img), dtype=np.uint8)
9
10 # б) плавний градієнт зі структурою
11 x_coords = np.linspace(0, 255, w_img)
12 y_coords = np.linspace(0, 255, h_img)
13 xx, yy = np.meshgrid(x_coords, y_coords)
14 img_gradient = np.clip((xx + yy) / 2.0, 0, 255).astype(np.uint8)
15
16 # в) низькоентропійне зображення (контрастні смуги / фігури)
17 img_pattern = np.zeros((h_img, w_img), dtype=np.uint8)
18 img_pattern[:64, :] = 50
19 img_pattern[64:, :] = 200
20
21 def calc_image_entropy(img):
22     hist, _ = np.histogram(img.flatten(), bins=256, range=(0, 256))
23     p = hist / img.size
24     h = -np.sum([pk * np.log2(pk) for pk in p if pk > 0])
25     r = 1.0 - h / h_max_img
26     return h, r, p
27
28 h_img1, r_img1, p_img1 = calc_image_entropy(img_noise)
29 h_img2, r_img2, p_img2 = calc_image_entropy(img_gradient)
30 h_img3, r_img3, p_img3 = calc_image_entropy(img_pattern)
31
32 print("\n--- Ентропійна оцінка 8-бітових зображень (M = 256 рівнів) ---")
33 print(f"1. Рівномірний шум: H = {h_img1:.4f} біт/піксель | Надлишковість R = {r_img1 * 100:.2f}%")
34 print(f"2. Плавний градієнт: H = {h_img2:.4f} біт/піксель | Надлишковість R = {r_img2 * 100:.2f}%")
35 print(f"3. Двохрівневий узор: H = {h_img3:.4f} біт/піксель | Надлишковість R = {r_img3 * 100:.2f}%")
36
37 plt.figure(figsize=(12, 6.5))
38 # зображення
39 plt.subplot(2, 3, 1)
40 plt.imshow(img_noise, cmap="gray", vmin=0, vmax=255)
41 plt.title(f"Шумове зображення\nH = {h_img1:.2f} біт/пікс")
42 plt.axis("off")
43
44 plt.subplot(2, 3, 2)
45 plt.imshow(img_gradient, cmap="gray", vmin=0, vmax=255)
46 plt.title(f"Градiєнтне зображення\nH = {h_img2:.2f} біт/пікс")
47 plt.axis("off")
48
49 plt.subplot(2, 3, 3)
50 plt.imshow(img_pattern, cmap="gray", vmin=0, vmax=255)
51 plt.title(f"Двохрівневий узор\nH = {h_img3:.2f} біт/пікс")
52 plt.axis("off")
53
54 # гістограми яскравості
55 plt.subplot(2, 3, 4)
56 plt.plot(p_img1, color="#1f77b4")
57 plt.fill_between(range(256), p_img1, color="#1f77b4", alpha=0.3)
58 plt.title(f"Гістограма (R = {r_img1*100:.1f}%)")
59 plt.xlabel("Яскравість (0..255)")
60 plt.ylabel("Імовірність P(k)")
61 plt.grid(True, linestyle="--", alpha=0.5)
62
63 plt.subplot(2, 3, 5)
64 plt.plot(p_img2, color="#2ca02c")
65 plt.fill_between(range(256), p_img2, color="#2ca02c", alpha=0.3)
66 plt.title(f"Гістограма (R = {r_img2*100:.1f}%)")
67 plt.xlabel("Яскравість (0..255)")
68 plt.grid(True, linestyle="--", alpha=0.5)
69
70 plt.subplot(2, 3, 6)
71 plt.plot(p_img3, color="#ff7f0e")
72 plt.fill_between(range(256), p_img3, color="#ff7f0e", alpha=0.3)
73 plt.title(f"Гістограма (R = {r_img3*100:.1f}%)")
74 plt.xlabel("Яскравість (0..255)")
75 plt.grid(True, linestyle="--", alpha=0.5)
76
77 plt.tight_layout()
78 plt.show()

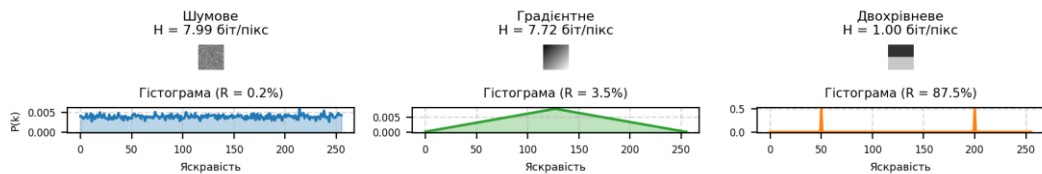
```

```

--- Порівняння ентропії для різних розподілів (M = 16 рівнів) ---
Максимальна ентропія H_max = log2(16) = 4.0000 біт
1. Рівномірний розподіл: H = 3.9986 біт | Надлишковість R = 0.03%
2. Нормальний розподіл: H = 3.0678 біт | Надлишковість R = 23.30%
3. Скошений розподіл: H = 2.7408 біт | Надлишковість R = 31.48%

--- Ентропійна оцінка 8-бітових зображень (M = 256 рівнів) ---
1. Рівномірний шум: H = 7.9879 біт/піксель | Надлишковість R = 0.15%
2. Плавний градієнт: H = 7.7212 біт/піксель | Надлишковість R = 3.48%
3. Двохрівневий узор: H = 1.0000 біт/піксель | Надлишковість R = 87.50%

```



Висновок:

У ході виконання лабораторної роботи було детально досліджено засади кількісної оцінки інформації в цифрових сигналах: алгоритми обчислення власної інформації подій, ентропії Шеннона, максимальної інформаційної ємності та коефіцієнта надлишковості джерела даних.

Аналіз логарифмічної міри Шеннона $I(E) = -\log_2 P(E)$ підтвердив, що кількість інформації є спадною функцією ймовірності: достовірні події ($P=1$) мають нульову інформаційну ємність $I=0$, а малоймовірні події несуть значний обсяг інформації. Експеримент із вибіркою $N=100$ для цифр 0-9 показав розкид інформації від 2.737 до 4.059 біт навколо теоретичного значення $-\log_2(0.1) = 3.322$ біт через статистичні флуктуації малої вибірки.

Дослідження поведінки ентропії $H(N)$ для розмірів вибірки від 50 до 5000 елементів підтвердило збіжність емпіричної ентропії до теоретичного максимуму $H_{\max} = \log_2(10) \approx 3.3219$ біт/символ за законом великих чисел. При $N=50$ надлишковість становила 4.36%, а при $N \geq 1000$ спала до часток відсотка ($<0.08\%$), що підтверджує стабільність оцінки.

Порівняльний аналіз розподілів виявив, що максимум ентропії ($H = 3.9986$ біт, $R = 0.03\%$ при $M=16$) досягається лише за рівномірного закону. Для гауссового та експоненційного розподілів надлишковість зростає до 23.30% та 31.48%. В оцінці 8-бітних зображень ($M=256$) білий шум практично не стискається ($H \approx 7.99$ біт/пікс, $R = 0.15\%$), тоді як структуровані узори мають надлишковість $R = 87.50\%$ ($H = 1.0$ біт/пікс), що визначає границі стиснення за теоремою Шеннона (коди Хаффмана, арифметичні, RLE).